JavaScript: Introduction

JavaScript

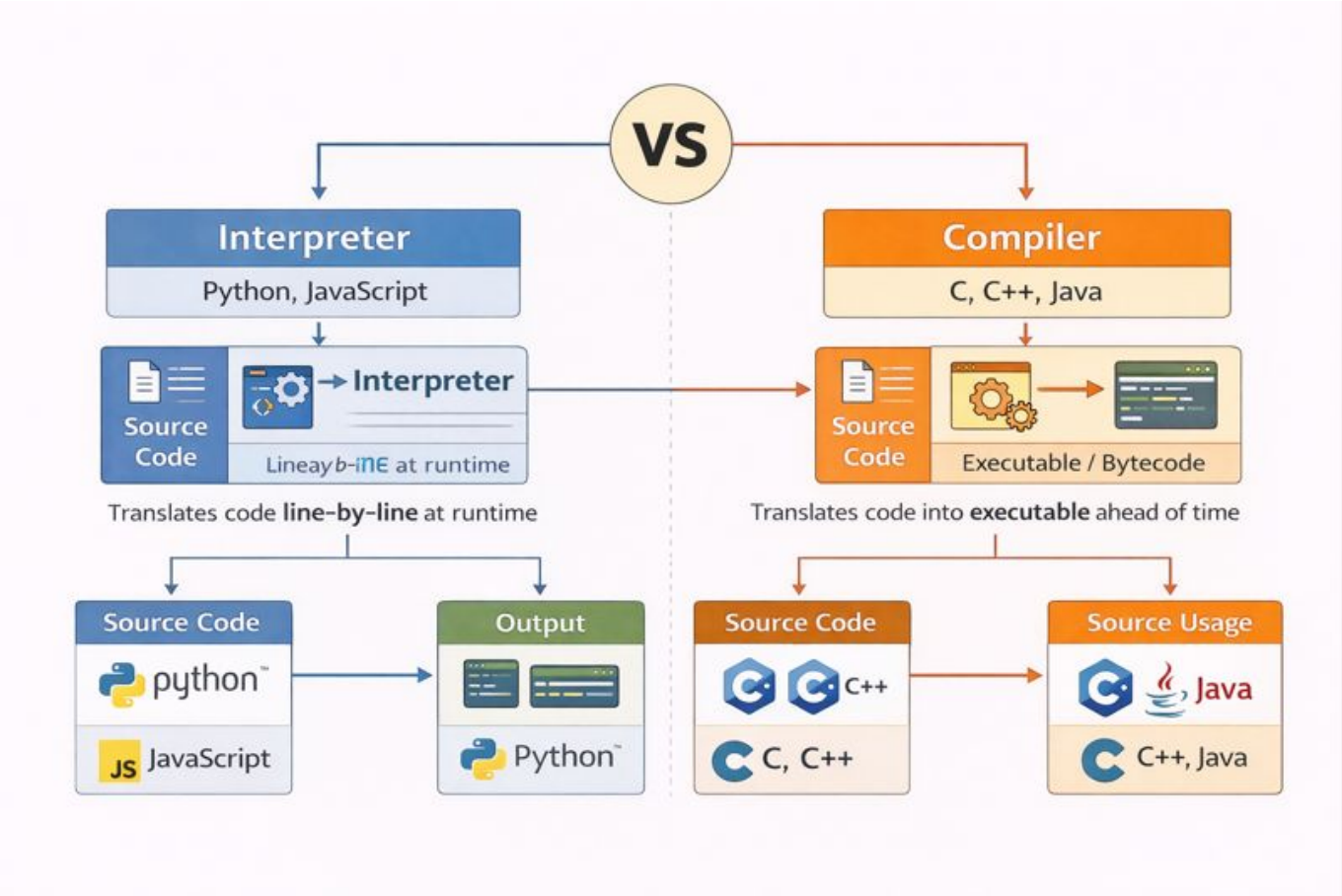
JavaScript is a high-level, interpreted programming language primarily used for:

- **Web development** (front-end and back-end with Node.js)
- **Mobile app development** (React Native, Expo)
- **Interactive web pages** and dynamic content

JavaScript vs Java: Interpreter vs Compiler

Interpreter vs Compiler

The fundamental difference between JavaScript and Java lies in how they execute code:



Interpreter (JavaScript)

An **interpreter** reads and executes code **line by line** at runtime:

How it works:

1. **Reads** the source code
2. **Translates** each line to machine code immediately
3. **Executes** the instruction right away
4. Moves to the next line and repeats

Compiler (Java)

A **compiler** translates the entire source code **before execution**:

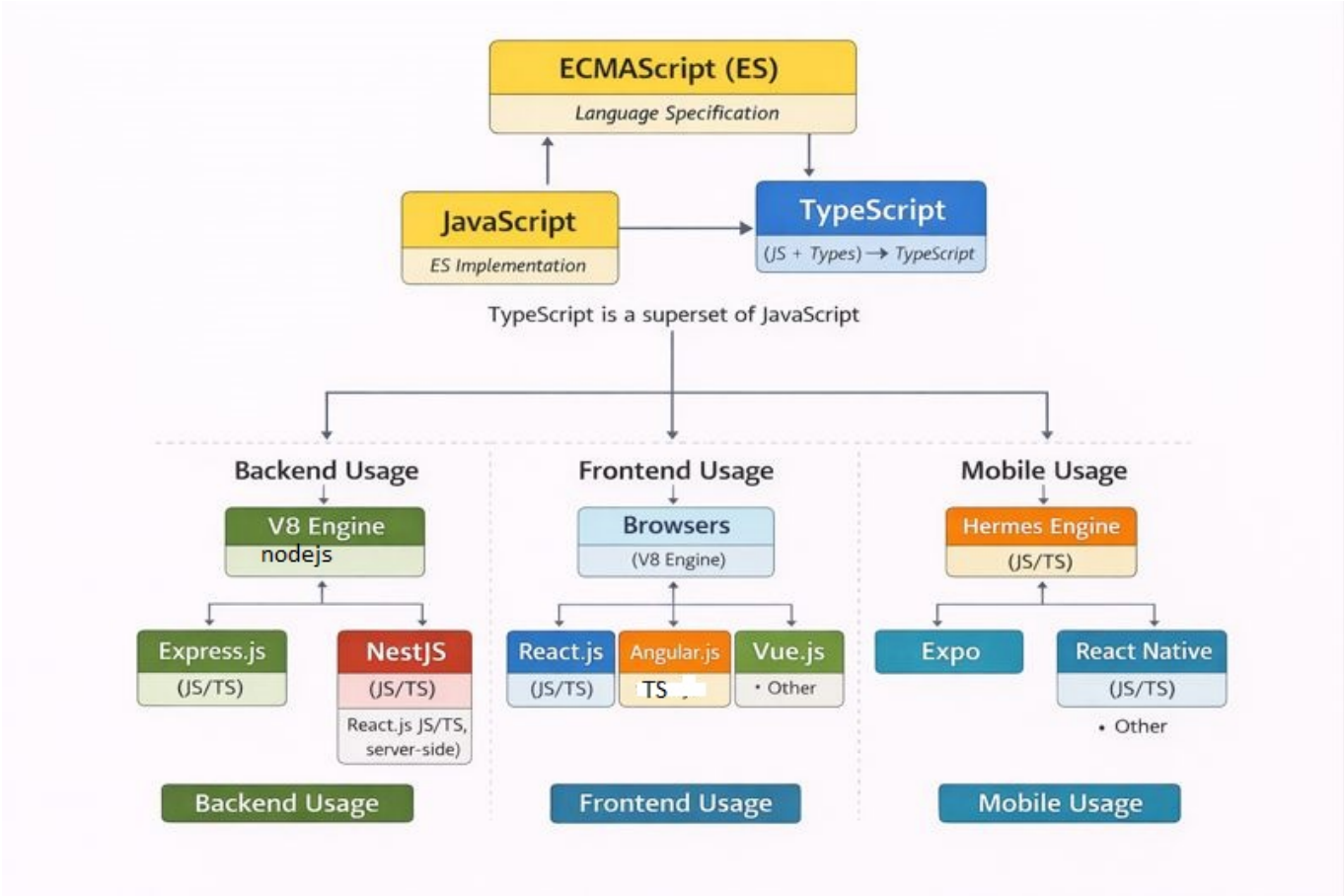
How it works:

1. **Reads** the entire source code
2. **Translates** all code to bytecode (for Java) or machine code
3. **Checks** for errors before execution
4. Creates an executable file or bytecode
5. The compiled code is then executed separately

Quick Comparison Table

Aspect	JavaScript (Interpreter)	Java (Compiler)
Execution	Line by line at runtime	Entire program before runtime
Speed	Slower execution, faster development	Faster execution, slower development
Errors	Found at runtime	Found at compile time
Platform	Browser/Node.js	JVM (Java Virtual Machine)

ECMAScript: JavaScript Standard



Key Points:

- **ECMAScript (ES)** is the official standard specification for JavaScript
- JavaScript is an implementation of the ECMAScript standard
- **ECMA International** maintains the standard (ECMA-262)
- New versions add features like:
    - ES6 (ES2015): Classes, arrow functions, let/const
    - ES2017: Async/await
    - ES2020: Optional chaining, nullish coalescing
- All modern browsers and Node.js implement ECMAScript standards

JavaScript Ecosystem: Key Technologies

TypeScript

- **Superset of JavaScript** with static typing
- Compiles to JavaScript before execution
- Catches errors at compile time
- Improves code maintainability and IDE support

ReactJS

- **Front-end library** for building user interfaces
- Component-based architecture
- Virtual DOM for efficient updates
- Used for web applications and SPAs

Node.js

- **JavaScript runtime** for server-side development
- Built on V8 engine
- Enables JavaScript to run outside browsers
- Used for back-end APIs, servers, and tools

Next.js

- **React framework** for production-ready web apps
- Server-side rendering (SSR) and static site generation (SSG)
- Built-in routing and optimization
- Full-stack React applications

V8 Engine

- **JavaScript engine** developed by Google
- Powers Chrome browser and Node.js
- Converts JavaScript to machine code
- Uses JIT (Just-In-Time) compilation for performance

Hermes

- **JavaScript engine** by Meta (Facebook)
- Optimized for React Native mobile apps

- Faster startup time and lower memory usage
- Pre-compiles JavaScript to bytecode

Expo

- **Framework and platform** for React Native development
- Simplifies mobile app development
- Provides tools, APIs, and services
- No native code required for many features